

第 10 节 实操任务 1 定义 KBState 状态

学习目标: `python -c "from workflows.state import KBState;`

`print(KBState.__annotations__)"` 输出 7 个字段

目标文件: `workflows/state.py`

步骤 1: 安装 LangGraph

```
1 pip install langgraph
```

验证安装成功:

```
1 python -c "import langgraph; print(f'langgraph {langgraph.__version__}' 3
2
3 ----
```

步骤 2: 创建 workflows 目录

```
1 cd ~/ai-knowledge-base
2 mkdir -p workflows
3 touch workflows/__init__.py
4
5 ----
```

步骤 3: 用 AI 编程工具生成 state.py

以下代码可以用 OpenCode、Claude Code、Cursor、Trae 或通义灵码等任意 AI 编程工具生成。

提示词:

```
1 请帮我编写 workflows/state.py, 定义 LangGraph 工作流的共享状态 :
2
3 需求 :
4 1. 使用 TypedDict 定义 KBState 类
5 2. 包含以下字段 :
6     - sources: list[dict] - 采集到的原始数据
7     - analyses: list[dict] - LLM 分析后的结构化结果
8     - articles: list[dict] - 格式化、去重后的知识条目
9     - review_feedback: str - 审核反馈意见
10    - review_passed: bool - 审核是否通过
11    - iteration: int - 当前审核循环次数 (最多 3 次)
12    - cost_tracker: dict - Token 用量追踪
13 3. 每个字段加中文注释说明用途和数据格式
```

生成的代码：（参考实现）

```

1  """LangGraph 状态定义 - AI 知识库工作流的核心数据结构
2
3  所有节点共享同一个 KBState, 通过 TypedDict 保证类型安全。
4  每个节点只修改自己负责的字段, 实现职责隔离。
5  """
6
7  from typing import TypedDict
8
9
10 class KBState(TypedDict):
11     """知识库工作流的全局状态
12
13     数据流向: sources → analyses → articles → review → save
14     review_loop 是本项目的核心教学点——展示如何用条件边实现质量门控。
15
16     Fields:
17         sources: 原始采集数据, 来自 GitHub API / RSS 等
18         analyses: 经 LLM 分析后的结构化结果
19         articles: 格式化、去重后的知识条目
20         review_feedback: 审核 Agent 的反馈意见 (中文)
21         review_passed: 审核是否通过
22         iteration: 当前审核循环次数 (最多 3 次)
23         cost_tracker: Token 用量追踪
24     """
25
26     sources: list[dict]          # 采集结果 (报告式: 摘要而非原始 HTML)
27     analyses: list[dict]         # 分析结果 (每条含 summary, tags, score)
28     articles: list[dict]         # 知识条目 (过滤 + 去重后的最终格式)
29     review_feedback: str         # 审核反馈 (具体改进建议, 非空表示需修改)
30     review_passed: bool          # 审核是否通过 (条件边的判断依据)
31     iteration: int               # 审核迭代次数 (>= 3 时强制通过)
32     cost_tracker: dict           # Token 统计 {prompt_tokens, completion_tokens}
33
34     ---

```

步骤 4：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

请解释 workflows/state.py 的设计：

1. 为什么用 TypedDict 而不是普通 dict？
2. 为什么 sources 是 list[dict] 而不是 str？
3. review_passed 为什么是 bool 不是 str？
4. 为什么需要 iteration 字段？

设计问题	答案
为什么用 TypedDict?	类型安全 + IDE 自动补全, LangGraph 要求 State 是 TypedDict
为什么 sources 是 list[dict]?	结构化数据方便下游解析, 不需要再次理解格式
为什么 review_passed 是 bool?	条件边需要布尔判断, 省去字符串解析
为什么需要 iteration?	防止审核循环无限执行, 提供循环出口
为什么 cost_tracker 是 dict?	灵活存储多种计费字段, 后续可扩展

步骤 5: 验证 KBState

```
1 python -c "  
2 from workflows.state import KBState  
3  
4 # 检查字段定义  
5 annotations = KBState.__annotations__  
6 print('KBState 字段: ')  
7 for name, type_hint in annotations.items():  
8     print(f' {name}: {type_hint}')9 print(f'\n共 {len(annotations)} 个字段')  
10  
11 # 创建一个实例  
12 state: KBState = {  
13     'sources': [],  
14     'analyses': [],  
15     'articles': [],  
16     'review_feedback': '',  
17     'review_passed': False,  
18     'iteration': 0,  
19     'cost_tracker': {},  
20 }  
21 print(f'实例创建成功, iteration = {state["iteration"]}')22 "
```

期望输出:

```
1 KBState 字段:  
2 sources: list[dict]  
3 analyses: list[dict]  
4 articles: list[dict]  
5 review_feedback: <class 'str'>  
6 review_passed: <class 'bool'>  
7 iteration: <class 'int'>
```

```
8     cost_tracker: <class 'dict'>
9
10 共 7 个字段
11 实例创建成功, iteration = 0
12
13 ---
```

步骤 6: 提交到 Git

```
1 git add workflows/__init__.py workflows/state.py
2 git commit -m "feat: add KBState definition for V3 LangGraph workflow"
3
4 ---
```

完成! 工作流的数据契约定义好了。KBState 的每个字段都是一份报告, 这就是报告式通信的代码体现。下一步实现节点逻辑。